

Arithmetic Logic Unit

COMS10015

Dr Tom Deakin

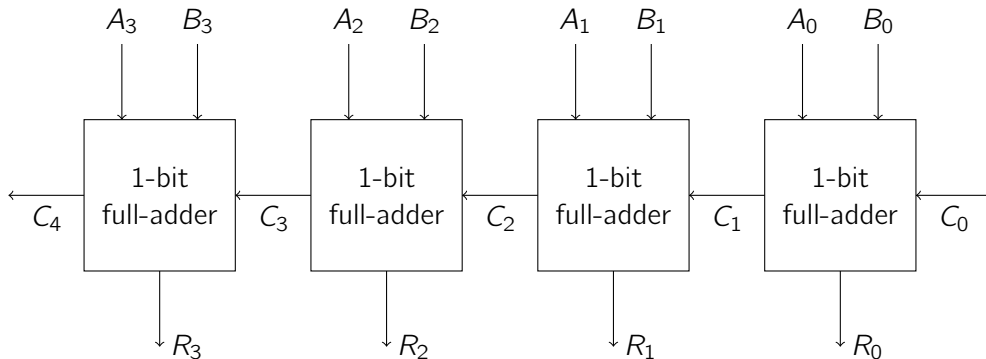
University of Bristol

Week 14

Adder output

Recall the ripple-carry full adder:

- ▶ 3 inputs: x , y and a carry-in ci
- ▶ Computes: $x + y + ci$
- ▶ Outputs: Sum and carry-out co



Adder-subtractor

Subtraction using addition

$$A - B = A + \bar{B} + 1$$

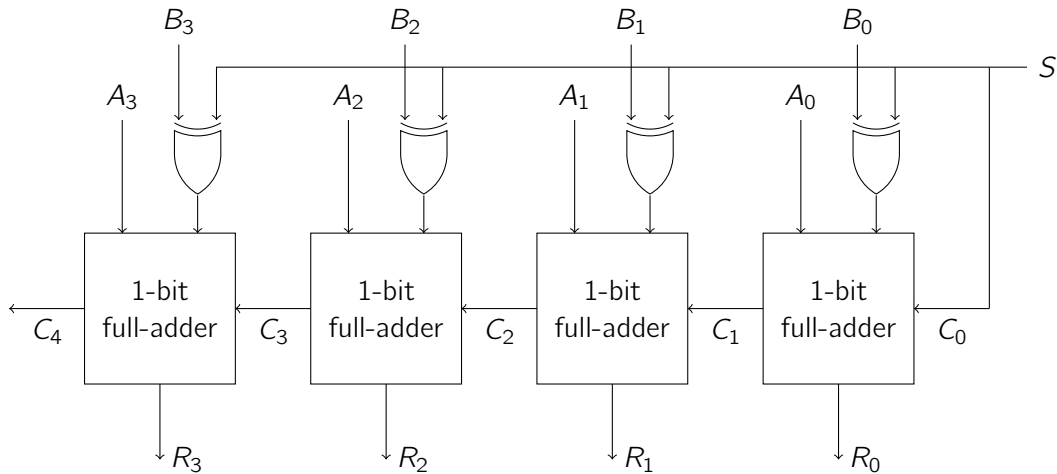
Apply two's complement to B with logical NOT and adding 1, then add to A .

Signal bit S used to control the circuit mode:

- ▶ $S = 0$ for addition $A + B$.
- ▶ $S = 1$ for subtraction.
 - ▶ Add 1 via $C_0 = 1$.
 - ▶ Invert B with XOR gate: $B_0 \oplus 1 = \text{NOT } B_0$.

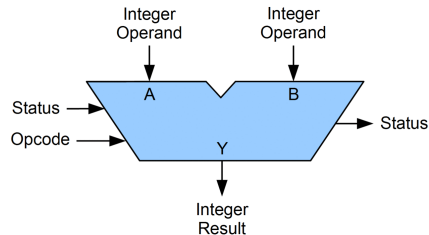
The adder-subtractor circuit can now perform two arithmetic operations on the inputs, **chosen with a control signal**.

Adder-subtractor: diagram



Arithmetic logic unit (ALU)

- ▶ Multi-function combinatorial logical circuit.
- ▶ Normally operates on integers.
- ▶ Typically implement functions such as:
 - ▶ Arithmetic operations: Addition, Subtraction.
 - ▶ Logical operations: AND, OR, XOR, NOT.
 - ▶ Bit shift operations.
- ▶ Control signals (parallel bus) used to select mode of operation.
- ▶ Carry in and carry out signals can chain multiple ALUs together to increase supported word length.
- ▶ Status output to report error, overflow, etc.



Picture from

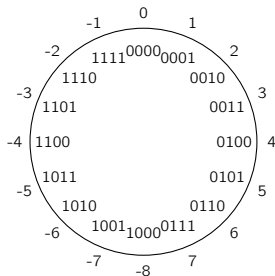
https://en.wikipedia.org/wiki/Arithmetic_logic_unit#/media/File:ALU_block.gif

Recap: Two's complement

Reminder:

In two's complement, for an n-bit integer i , $i + -i = 2^n = 0$.

To convert an integer i from one's complement to $-i$ in two's complement, invert the bits and add 1.



"Clock face" recreated from <https://medium.com/@anne75/twos-complement-679c0e60c4b2>

Two's complement overflow

Overflow will occur when:

- ▶ $+ve + +ve = -ve$
 - ▶ e.g., $6 + 3 = -7$ for 4-bit integers
- ▶ $-ve + -ve = +ve$
 - ▶ e.g., $-2 + -8 = 6$ for 4-bit integers

How can we spot this?

Two's complement overflow

- ▶ In Two's complement, most-significant bit represents the sign.
- ▶ Overflow happens when add sign bits and get a different sign bit.

Two's complement overflow

- ▶ In Two's complement, most-significant bit represents the sign.
- ▶ Overflow happens when add sign bits and get a different sign bit.
- ▶ Two cases where result bit is not correct.
Why?

1-bit full-adder truth table

ci	x	y	co	r
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Two's complement overflow

- ▶ In Two's complement, most-significant bit represents the sign.
- ▶ Overflow happens when add sign bits and get a different sign bit.
- ▶ Two cases where result bit is not correct. Why?
- ▶ Carry in to the sign bit also involved in addition.
- ▶ **Notice** carry in and carry out are **different** *only* for these cases.
- ▶ Therefore, compare final two carry bits for overflow.
- ▶ Note, can have a valid carry out without overflow (i.e., last 3 rows of truth table).

1-bit full-adder truth table

ci	x	y	co	r
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Two's complement overflow

4-bit two's complement has range $[-8, +7]$ ($[-2^{N-1}, 2^{N-1} - 1]$).

Example, compute $6 + 3 = 9 > 7$ which will overflow:

```
  0110 (6)
  0011 (3)
  ----
  1001 (-7)
  0110 (co)
```

The final two carry out bits do not match:
overflow!

1-bit full-adder truth table

ci	x	y	co	r
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Note where $ci = 1$ and x and/or y is 1,
 $co = 1$ (“carry bit is sticky”)

Status registers

As well as carry-out and overflow, ALU might report additional information about result:

- ▶ Whether result is zero.
- ▶ Whether result is negative.
- ▶ Whether result is even/odd.
- ▶ ...

This status often stored in a special-purpose register:

x86 16-bit FLAGS register.

Arm 32-bit Current Program Status Register.

ISA Status Register

The flags are defined and used in ISA.

Example: x86 integer subtraction

SUB: Subtract

$$dst \leftarrow dst - src$$

Sets six different flags, including *CF* carry flag if carry-out from most-significant bit.

SBB: Integer Subtraction with borrow

$$dst \leftarrow dst - (src + CF)$$

Instruction **uses** the carry flag.

Hex 8 Module(sim) ALU

Already been using a 4-bit ALU in Modulesim.

- ▶ Performs addition, subtraction, input pass through, and logical comparisons.
- ▶ Has two outputs: arithmetic result and logical result.

Use control signal to choose operation mode:

Control	Arithmetic output	Logical output
**00	$areg + breg$	$areg + breg == 0$
**01	$areg$	$areg == 0$
*110	$areg - breg$	$areg == breg$
*111	$areg - breg$	$areg < breg$

Bit 2 is the carry bit.

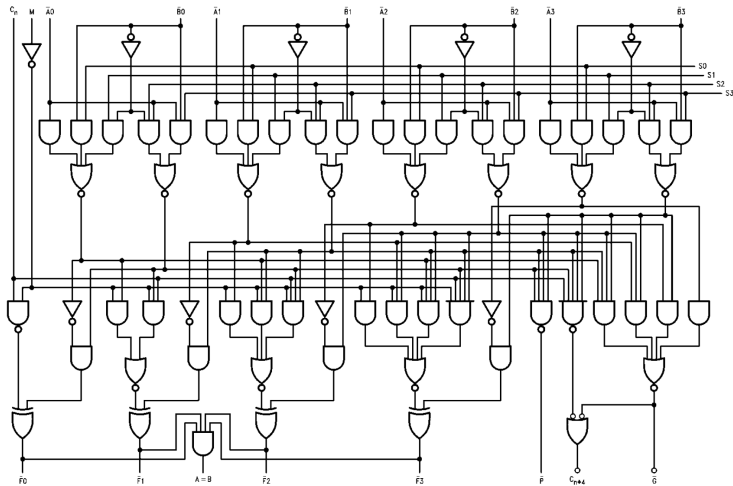
Example: Texas Instruments 74181 ALU

Texas Instruments 74181 ALU

4-bit ALU:

- ▶ 4-bit function signal, plus 1-bit mode selector (arithmetic/logical).
- ▶ 16 ALU operations: add, subtract, double, ...
- ▶ 16 logical operations: XOR, compare, AND, NAND, OR, NOR, ...
- ▶ Two 4-bit inputs, one 4-bit output.
- ▶ Outputs for ripple carry and look ahead carry and equality.

74181 circuit diagram



From tech sheet: <http://ee-classes.usc.edu/ee459/library/datasheets/DM74LS181.pdf>

74181 functions (1)

Mode select				Logic	Arithmetic
S3	S2	S1	S0	H=1	H=0
0	0	0	0	$\neg A$	A
0	0	0	1	$\neg A \vee \neg B$	$A \vee B$
0	0	1	0	$\neg A \wedge B$	$A \vee \neg B$
0	0	1	1	0	-1
0	1	0	0	$\neg(A \wedge B)$	$A + (A \wedge \neg B)$
0	1	0	1	$\neg B$	$(A \vee B) + (A \wedge \neg B)$
0	1	1	0	$A \oplus B$	$A - B - 1$
0	1	1	1	$A \wedge \neg B$	$(A \wedge B) - 1$

No carry: $C_N = 1$.

Carry: $C_N = 0$, adds 1 to Arithmetic result.

74181 functions (2)

Mode select				Logic	Arithmetic
S3	S2	S1	S0	H=1	H=0
1	0	0	0	$\neg A \vee B$	$A + (A \wedge B)$
1	0	0	1	$\neg A \oplus \neg B$	$A + B$
1	0	1	0	B	$(A \vee \neg B) + (A \wedge B)$
1	0	1	1	$A \wedge B$	$(A \wedge B) - 1$
1	1	0	0	1	$A + A$
1	1	0	1	$A \vee \neg B$	$(A \vee B) + A$
1	1	1	0	$A \vee B$	$(A \vee \neg B) + A$
1	1	1	1	A	$A - 1$

No carry: $C_N = 1$.

Carry: $C_N = 0$, adds 1 to Arithmetic result.

Multiplication

Repeated addition

- ▶ Not yet seen a circuit for multiplication.
- ▶ Naive general approach is to use repeated addition.
- ▶ E.g. $4 \times A = A + A + A + A$
- ▶ If multiply by a known constant, processor could just execute fixed number of **ADD** instructions.
- ▶ Compiler could generate a loop (counter and branch) for the general case (see Compiler lecture).
- ▶ **But** run time is data dependent!

Need a better approach — design a circuit to perform multiplication.

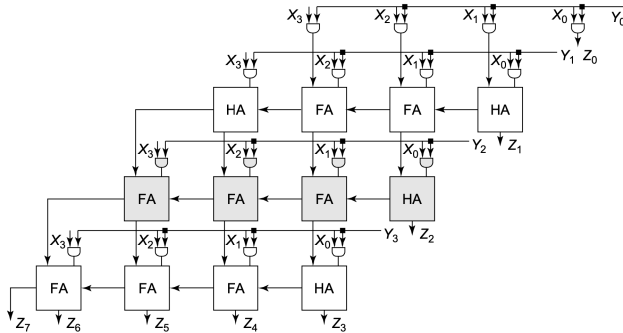
Multiplication by shifts and adding

- ▶ Same process as long multiplication.
- ▶ $17 \times 11 = (17 \times 1 \times 10^0) + (17 \times 1 \times 10^1) = 17 + 170 = 187$
- ▶ Multiply first number by each digit of second number, shifted by place value.
- ▶ Sum up the partial results.
- ▶ Simple in binary: AND, left-shift and addition.
- ▶ For a fixed word size, fixed number of additions.
 - ▶ I.e., can implement with layers of adders.
- ▶ For n-bit integers, result is 2n-bit integer.
- ▶ Only consider unsigned integers, but can extend idea to signed.

Multiplication by shifts and adding

```
      0111 (7)
x     1011 (11)
-----
      0111 (1 AND 0111)
     0111  (1 AND 0111, shift 1 place)
    0000   (0 AND 0111, shift 2 places)
+ 0111    (1 AND 0111, shift 3 places)
----- (add)
 01001101 (77)
 0111110  (carry)
```

Multiplication circuit



© Digital Integrated Circuits^{2nd}

Arithmetic Circuits

Note Y shifts are implicit based on AND/adder inputs.

Picture from <http://www.micro.deis.unibo.it/~baccaran/Rabaey/chapter11.pdf>

Wallace Tree

- ▶ In 1964, Australian computer scientist Chris Wallace showed an improvement.
- ▶ Reduces the number of steps needed to sum partial results.
- ▶ Reduces the propagation delay (latency) of the circuit (no need to wait for carry like in the ripple-adder).
- ▶ Notice a full-adder computes the sum of *three* bits, producing two output bits:
 - ▶ One output bit for current position.
 - ▶ One output bit (carry-out) for next higher position.
- ▶ Combine groups of three summands (partial products) using full-adders;
- ▶ If only two summands, combine with half-adder.
- ▶ Outputs flow down to next layer (output as new input in same position, carry-out as new input in next position).
- ▶ Repeat adding layers until at most two inputs in each position.

Constructing a Wallace Tree

- ▶ Compute all the multiplications by AND'ing each bit with every other (as before).
- ▶ If any position has more than 3 digits to sum, add a layer to the tree.
- ▶ Construct a layer by grouping digits of summands:
 - ▶ Use full-adders to combine groups of 3 summands.
 - ▶ If only 2 summands, use half-adders to combine instead.
 - ▶ Pass remaining summands through to the next stage.
- ▶ Finish when only 2 partial sums to accumulate, so use regular n -bit (ripple carry) adder.

Position i refers to the digits with weights 2^i .

Wallace tree example (1)

Step 1: compute partial sums

```
      0111 (7)
x     1011 (11)
-----
      0111
      0111
      0000
      0111
      -----
```

			0	1	1	1
		0	1	1	1	
	0	0	0	0		
0	1	1	1			

Need to sum down each column (position), which corresponds to each digit of solution.

Wallace tree example (2)

Stage 1: One position has more than three wires remaining, so add layer to sum groups of three:

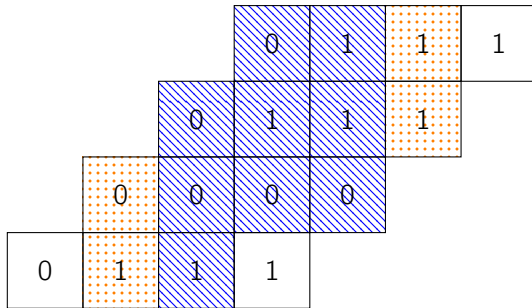
			0	1	1	1
		0	1	1	1	
	0	0	0	0		
0	1	1	1			

Key:



Wallace tree example (3)

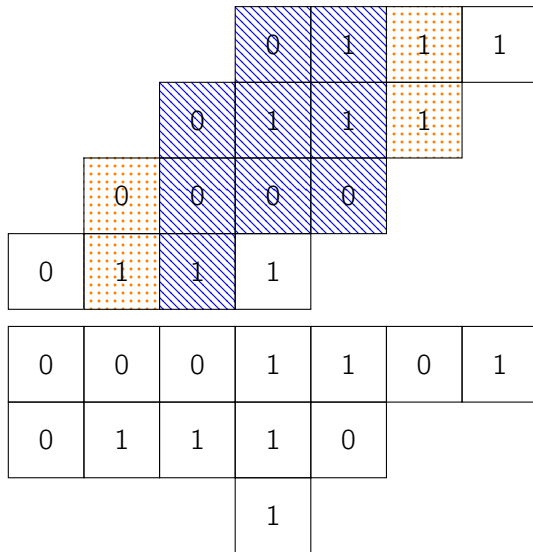
Stage 1: One position has more than three wires remaining, so add layer to sum groups of three:



Key:



Wallace tree example (4)



Wallace tree example (5)

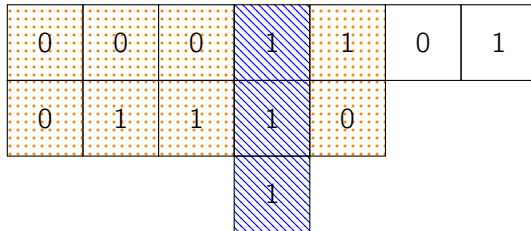
Stage 1: Result

0	0	0	1	1	0	1
0	1	1	1	0		
			1			

Remember, each adder produces **two** outputs: one for its position, one for the next higher position (carry-out).

Wallace tree example (6)

Stage 2: Take results from above layer, and add another layer as at least one position has 3 wires.



Key:



Wallace tree example (7)

0	0	0	1	1	0	1
0	1	1	1	0		
			1			

0	0	0	1	0	1	0	1
	0	1	1	1			

Wallace tree example (8)

Stage 2: Result

0	0	0	1	0	1	0	1
	0	1	1	1			

Wallace tree example (8)

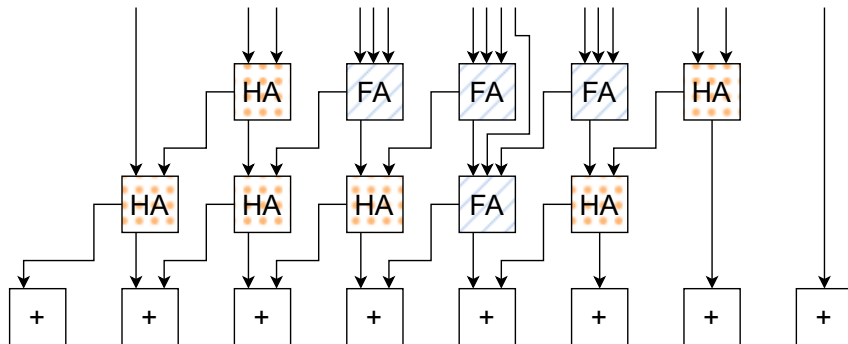
Stage 2: Result

0	0	0	1	0	1	0	1
	0	1	1	1			

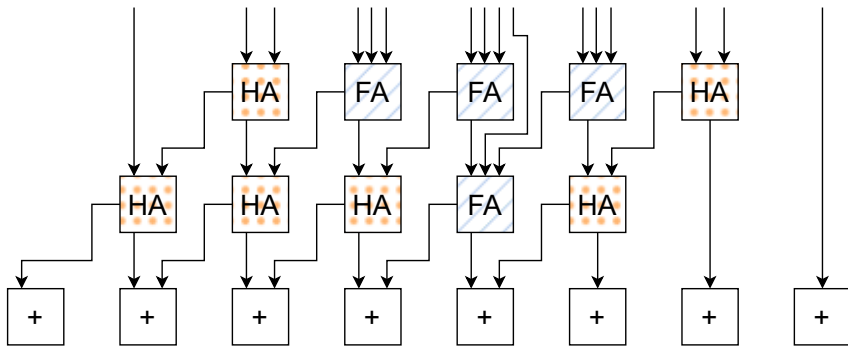
Now there are only 2 wires in each position: use 8-bit ripple-carry full-adders to finish summation.

0	1	0	0	1	1	0	1
---	---	---	---	---	---	---	---

4-bit Wallace Tree



4-bit Wallace Tree



Notice how the propagation delay is reduced vs the shift-adder approach we saw earlier: longest path is two full-adders and the ripple-carry.

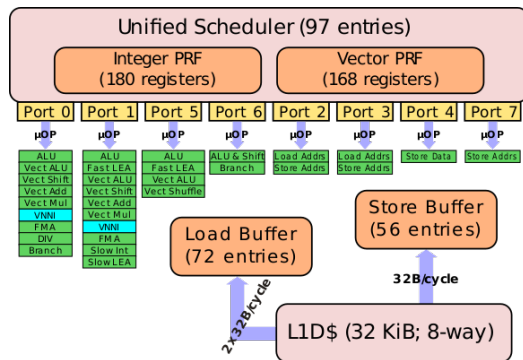
More multiplication circuits

- ▶ A Dadda Tree multiplier is similar to a Wallace Tree but optimise for number of adders (rather than reducing all groups of three).
- ▶ Booth encoding (look for small groups of 1s) to reduce number of partial products (before the Tree accumulator); helps support Two's complement.
- ▶ Parallel summation at the final layer (e.g., Kogge-Stone carry-lookahead).
- ▶ Support for pipelining *within* the ALU itself (for multi-cycle operations).
- ▶ Approaches for floating-point data types (sign, exponent, mantissa).
- ▶ ...

Other arithmetic functions

Fixed function units

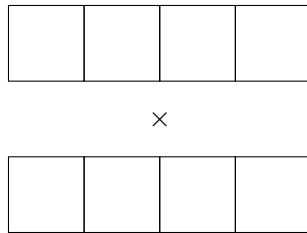
- ▶ Block diagram of the Cascade Lake micro-architecture.
- ▶ Lots of different Execution Units, each with different function.
- ▶ E.g. ALU, MUL, DIV/SQRT, ...
- ▶ More exotic things such as FMA, AES, Vector operations, ...
- ▶ Cascade Lake adds specific convolution instructions to support Neural Networks.



Picture from https://en.wikichip.org/wiki/File:casadeLake_scheduler.svg

Same Instruction, Multiple Data (SIMD)

- ▶ Want to perform same arithmetic instruction on different data.
- ▶ Exploit data level parallelism.
- ▶ One instruction does same operation on each element of vector.
- ▶ Can design vector units in micro-architecture, and vector instructions in ISA for this.
 - ▶ e.g. x86 AVX, Arm Neon, Arm SVE, ...
- ▶ Many modern processors have support for vector operations.
- ▶ Take COMS30046 Advanced Computer Architecture and/or COMS30053 HPC for more on this.



Further Reading

- ▶ L20. SIMD Processors: <https://safari.ethz.ch/digitaltechnik/spring2018/doku.php?id=schedule>
- ▶ <https://en.wikipedia.org/wiki/74181>
- ▶ 74181 LogiSim implementation: <https://github.com/stsvetkov/L8cpu>
- ▶ More detail on 74181: <http://www.righto.com/2017/03/inside-vintage-74181-alu-chip-how-it.html>
- ▶ Data Sheet
<http://ee-classes.usc.edu/ee459/library/datasheets/DM74LS181.pdf>
- ▶ Arithmetic:
<https://www.doc.ic.ac.uk/~eedwards/compsys/arithmetic/index.html>
- ▶ Adder-subtractor: <https://en.wikipedia.org/wiki/Addersubtractor>
- ▶ Arithmetic circuits:
<http://www.micro.deis.unibo.it/~baccaran/Rabaey/chapter11.pdf>

Further Reading

- ▶ Wallace Trees: https://en.wikipedia.org/wiki/Wallace_tree
- ▶ C.S. Wallace, A Suggestion for a Fast Multiplier,
<https://doi.org/10.1109/PGEC.1964.263830>
- ▶ Multiplication <http://web.mit.edu/6.111/www/f2017/handouts/L08.pdf>
- ▶ Hennessy & Patterson: Computer Architecture: A Quantitative Approach,
Online Appendix J, Computer Arithmetic by D. Goldberg.
- ▶ Patterson & Hennessy: Computer Organization and Design, Chapter 3.3.